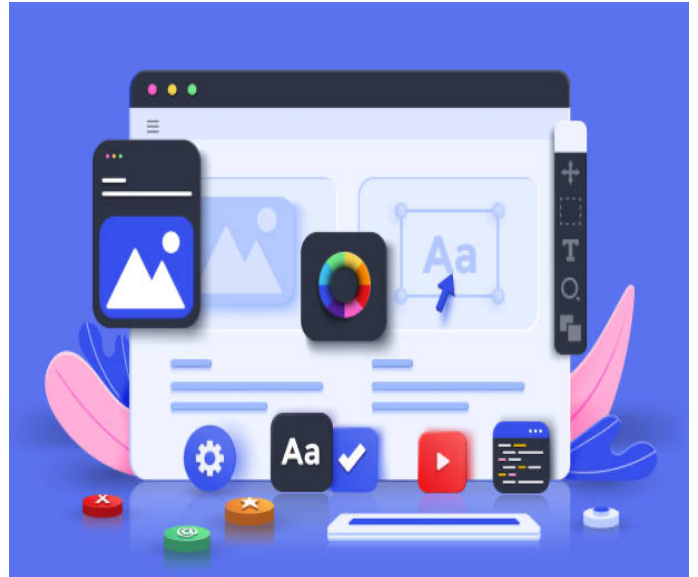


SD

—



Module Mijn eerste webgame

Instructie 03 – Spel gebruiksklaar maken (Initialiseren)

- JavaScript bestand maken
- Spel initialiseren

Auteur: Johan Strootman

Afkorting: SMN

Versie: juli 2023

Inhoudsopgave

Inhoudsopgave	3
MAPPENSTRUCTUUR VAN ONS PROJECT	5
Inleiding.....	5
Hoe ziet onze projectmap eruit?	5
JAVASCRIPT BESTAND AANMAKEN EN KOPPELEN.....	6
Stap 1 – Bestand aanmaken in de juiste map.....	6
Stap 2 – Bestand koppelen aan onze webpagina	6
SPEL GEBRUIKSKLAAR MAKEN	7
Inleiding.....	7
Hoe doen we dat?	7
TOOLS: INSPECTOR & CONSOLE.....	8
Inleiding.....	8
Hoe open je de Inspector?	8
Stap 1.....	8
Stap 2	9
Stap 3.....	9
Tot slot	9
SETUP VAN DE GAME.....	10
Inleiding.....	10
Wat maken we op uit bovenstaande afbeelding?	10
Globale variabelen aanmaken	11
Voor welke elementen hebben we variabelen nodig?	11
Zijn er nog andere globale variabelen nodig?	11

Welke dan?	11
Het resultaat.....	12

MAPPENSTRUCTUUR VAN ONS PROJECT

Inleiding

We maken als 'Best Practice' een aparte projectmap aan in de root van ons lokaal geïnstalleerde webserver (XAMPP/WAMP).

In ons projectmap programmeren we alles, maar we maken ook daar weer een onderverdeling in mappen en doen daarbij net of dit de root is van een echte online webserver. Zo behandelen we overigens ieder project lokaal op onze computer. We simuleren steeds, door a) een projectmap aan te maken en b) de projectmap weer onder te verdelen in submappen, een echte webserver voor ons project.

Hoe ziet onze projectmap eruit?



Figuur 1 - Onze mappenstructuur

Wij maken dus voor ons project bovenstaande mappen en bestanden structuur aan.

JAVASCRIPT BESTAND AANMAKEN EN KOPPELEN

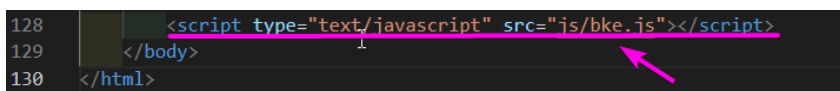
Stap 1 – Bestand aanmaken in de juiste map

In de map **js** maken we een nieuw bestand aan met de naam **bke.js**

Stap 2 – Bestand koppelen aan onze webpagina

In het bestand **index.html** plaatsen we vlak boven de sluittag van BODY de volgende SCRIPT-tag:

```
128 <script type="text/javascript" src="js/bke.js"></script>
129 </body>
130 </html>
```



Figuur 2 - Script-tag toevoegen

SPEL GEBRUIKSKLAAR MAKEN

Inleiding

Wat bedoelen we met '*gebruiksklaar maken*'?

Daarmee bedoelen we dat ons programma eerst een aantal taken moet uitvoeren zodat het spel ook echt gespeeld kan worden.

Dit noemen we ook de **INITIALISATIE** of de **SETUP** van ons programma.

Hoe doen we dat?

Dit doen we door de browser te vertellen dat een bepaald stuk code direct moet worden uitgevoerd zodra de browser klaar is met het verwerken van de webpagina in het DOM.

We gebruiken het internet om op te zoeken hoe we dit kunnen gaan doen. Daarbij is één ding belangrijk, namelijk we gebruiken altijd **Engelstalige zoektermen**. Dit doen we omdat een programmeertaal ook Engelstalig is.

Hoe weten we dat de browser klaar is met laden en verwerken?

De browser vertelt dit door een **event** (gebeurtenis) te **triggeren** (activeren). Dit is de zogenaamde **LOAD** event.

In JavaScript kunnen we aan dit event een functie koppelen met daarin de code die dan moet worden uitgevoerd.

TOOLS: INSPECTOR & CONSOLE

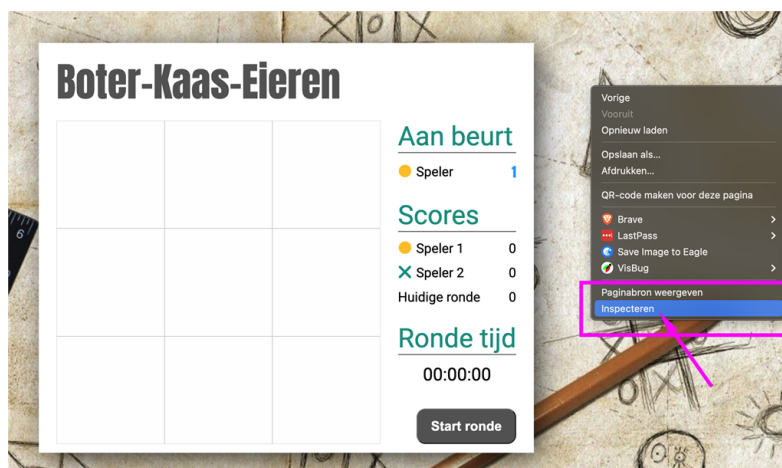
Inleiding

Een **standaard tool** die een JavaScript developer altijd open heeft staan is de **inspector** van de browser. Met de inspector kun je van iedere pagina de HTML-code zien en aanpassen, CSS-code zien en aanpassen, maar ook de Console openen om a) foutmeldingen te zien die ontstaan zijn door b.v. je eigen JavaScript-code en b) om handmatig JavaScript-code in te tikken om uit te proberen b.v.

Hoe open je de Inspector?

De inspector is een tool in de browser die we als web developer veel zullen gebruiken.

Stap 1



Figuur 3 - Rechts klikken en dan kiezen voor Inspecteren

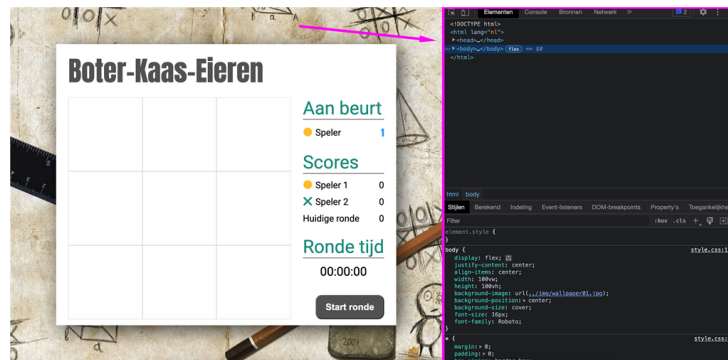
Als je met de rechter muisknop klikt ergens op de pagina dan open je een pop-up venster met onderin de keus Inspecteren (Let op: Bij iedere browser kan dit weer iets anders genoemd worden, maar over het algemeen iets met inspector of inspecteren).

Klik op de keus Inspecteren.

Stap 2

Er opent nu een nieuw gedeelte in je browser venster, maar let op soms kan dit ook een apart venster zijn.

Hieronder zie je een voorbeeld van hoe dit er uit zou kunnen zien, want in het voorbeeld hieronder staat het nieuw geopende deel rechts. Maar je kan zelf instellen waar dit deel voortaan geplaatst moet worden.

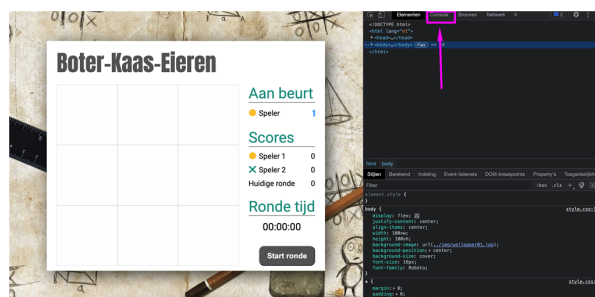


Figuur 4 - Inspector geopend

In het deel wat je nu ziet kun je de HTML-opbouw van de geopende webpagina zien en daaronder de geldende CSS styling.

Stap 3

Om de console te openen en dus JavaScript meldingen te zien of JavaScript code in te tikken klik je boven in dit deel op **Console**.



Figuur 5 - Console openen

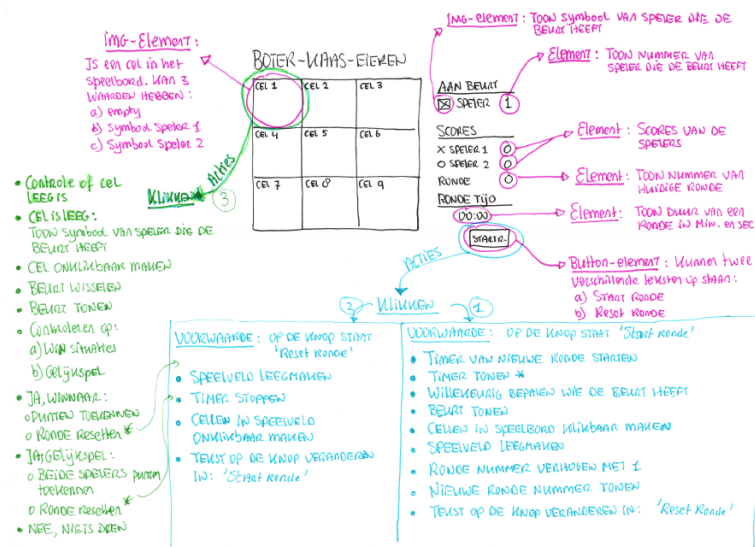
Tot slot

Als je een JavaScript bestand aan de webpagina hebt gekoppeld en in dit JavaScript b.v. variabelen hebt aangemaakt dan kun je in de console ook gewoon bij de variabelen er dus gewoon mee werken. Dit geldt ook voor functies die jezelf hebt gemaakt in je code.

SETUP VAN DE GAME

Inleiding

In het voorbereidend werk hebben we beschreven wat ons spel/programma moet doen om ons spel speelklaar te maken.



Figuur 6 - Het voorwerk, uitgangsbasis voor de setup

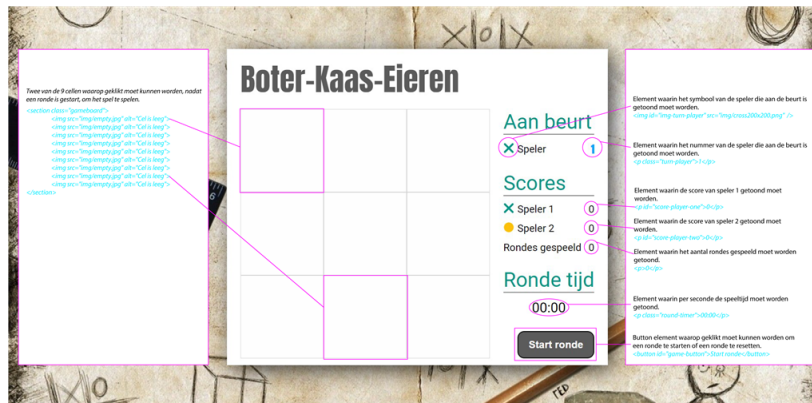
Wat maken we op bovenstaande afbeelding?

- We zien dat ons spel pas kan worden gespeeld als één van beide spelers op de knop klikt. Dit betekent dat we sowieso de knop klikbaar moeten maken.
- We zien dat er een aantal elementen op de pagina zijn die we in onze code moeten 'binnenhalen' omdat we er gedurende het spel iets mee moeten doen. Zoals het element op de pagina waar het nummer van de speler moet staan die aan de beurt is.
- We zien dat we alle cellen in het speelveld moeten binnenhalen, want deze moeten we later klikbaar en niet klikbaar kunnen maken.
- We kunnen ook de conclusie trekken dat alle elementen moeten worden opgeslagen in variabelen in JavaScript. En dat deze variabelen globaal moeten worden gemaakt. Want alleen variabelen die globaal zijn kunnen overal in onze code gebruikt worden.

Globale variabelen aanmaken

Voordat we gaan werken aan programma code die moet worden uitgevoerd bij het LOAD event willen we eerst de nodige variabelen aanmaken en alvast vullen (initialiseren) met een standaardwaarde. Want pas daarna kan de te programmeren code werken.

Voor welke elementen hebben we variabelen nodig?



Figuur 7 - Elementen in kaart gebracht

Hierboven hebben we met aantekening de elementen in kaart gebracht waarmee we in JavaScript iets moeten doen. Voor deze elementen maken we dus globale variabelen aan.

Zijn er nog andere globale variabelen nodig?

Ja.

Welke dan?

Hoe houden we bijvoorbeeld de rondetijd bij? Dat kunnen we niet rechtstreeks in het bijbehorende element doen. We hebben dus nog een zogenaamde **hulpvariabele** nodig, deze wordt dan per seconde bijgewerkt met de juiste tijd

Om de rondetijd ook weer te kunnen stoppen moeten we de lopende 'klok' ook ergens opslaan. Ook hiervoor hebben we een **hulpvariabele** nodig.

Hoe zit het met de scores? Ook voor de scores van beide spelers, want ook dit kunnen we niet rechtstreeks in de bijbehorende elementen doen.

Hoe zit het met het nummer van de speler die aan de beurt is? Hetzelfde als voor de scores.

Het resultaat

Allereerst, we geven altijd **duidelijke en beschrijvende namen** aan de variabelen. Als we dat doen kunnen we aan de hand van de naam van een variabele al zien wat het gegeven of element is die erin is opgeslagen.

Vervolgens zijn de name altijd in **kleine letters** en volgen we het principe **snake-case**.

Snake-case wil zeggen dat het compleet in kleine letters is en we verschillende woorden in een naam van elkaar scheiden met een underscore (`_`) oftewel liggend streepje.

We voorzien de variabelen ook van **commentaar** waarin we ook in de code nog eens toelichten waarvoor de betreffende variabele dient. Dit is 'Best Practice'.

```
7 // -----
8 // GLOBALE VARIABLEN
9 // -----
10
11 // ELEMENTS
12 let element_turn_number; // Tonen van het nummer van de speler die aan de beurt is
13 let element_turn_image; // Tonen van de afbeelding van de speler die aan de beurt is
14 let element_score_player1; // Tonen van de score van speler 1
15 let element_score_player2; // Tonen van de score van speler 2
16 let element_round_time; // Tonen van de ronde tijd
17 let element_rounds_played; // Tonen van de rondes gespeeld tot nu toe
18 let element_cells; // De cellen in het spelbord, hierin tonen we de afbeelding van een speler
19 let element_game_button; // De knop om een ronde te starten of te resetten
20
21 // HELPER VARIABLES
22 let current_player = 1; // Het nummer van de speler die aan de beurt is
23 let score_player1 = 0; // Hier houden we de score van speler 1 bij
24 let score_player2 = 0; // Hier houden we de score van speler 2 bij
25 let current_round = 0; // Welke ronde is het
26 let timer_id; // ID van de interval, dit is de klok van een ronde
27 let elapsedTimeInSeconds; // Verlopen ronde tijd in seconden
28
```

Figuur 8 - Resultaat: Alle benodigde globale variabelen